ChatGPT said:

Yes. Based on the **previous practical question, the Project Tracking Agent folder, Unit 4, the Custom GPT files, the demo inputs/outputs, and the real-world prompt examples**, this is what I would study for tomorrow.

Think of this as your **Practical Exam Survival Sheet**.

---

1. First understand what the practical is testing

I do **not** think you will be expected to build a huge AI program from scratch.

I think you may be given:

- a scenario,
- an Agentic AI program,
- some `.md`, `.csv`, `.txt` or similar files,
- a knowledge-base folder,
- and instructions/questions.

Then you may have to:

RUN → STUDY → ADD → TEST → GENERATE → PROMPT → EXPLAIN

That is the practical.

Unit 4 says an agent basically needs a **model, tools and instructions**, and the instructions should clearly explain what the agent should do. Unit 4 - A-Practical-Guide-to-B…

---

2. The most important formula to remember

Write this on rough paper when the exam starts:

> **R G D T K G O S**

Meaning:

Letter	Meaning	Question to ask yourself
R	Role	Who is the agent?
G	Goal	What must it achieve?

Letter	Meaning	Question to ask yourself
D	Data/Input	What information does it use?
T	Tasks	What must it do?
K	Knowledge	Which supporting files does it need?
G	Guardrails	What must it NOT do?
O	Output	What should it produce?
S	Stop/Escalate	When does it stop or ask a human?

If you can answer these eight things, you can handle many scenarios.

---

3. ROLE trick

Look at **who the agent is supposed to help or what job it performs**.

If the scenario says:

> The university wants an AI system to analyse student performance.

Role:

> You are a **Student Performance Agent**.

If it says:

> A company wants to track projects.

Role:

> You are a **Project Tracking Agent**.

If it says:

> HR wants to screen CVs.

Role:

> You are a **Recruitment Screening Agent**.

Easy formula:

> **You are a [scenario topic] agent.**

---

4. GOAL trick

Ask:

> **What is the final thing they want the agent to achieve?**

Example:

Scenario:

> Identify students who need support.

Goal:

> Your goal is to identify students who may need academic support.

Scenario:

> Detect delayed project tasks.

Goal:

> Your goal is to identify project delays and risks.

Formula:

> **Your goal is to [main purpose].**

---

5. DATA / INPUT trick

This is one of the things you asked about.

Ask:

> **What information is given to the agent?**

Those things become your **DATA**.

Examples:

Scenario	DATA
Student performance	marks, attendance, assignments
Project tracking	task register, deadlines, weekly updates
Recruitment	CVs, job requirements
Expenses	expense records, company policy
Inventory	stock records, sales data
Customer service	customer query, account details
Marketing	campaign data, engagement data
Sales	sales records, customer data
IT	incident logs, alerts
Vendors	supplier records, performance metrics

So if the question says:

> Analyse student marks and attendance...

Then you already know:

DATA = student marks and attendance records.

---

6. PROBLEM trick — IDAM-CRR

You already learned:

> **IDAM-CRR**

I = Identify
D = Detect
A = Analyse
M = Monitor
C = Compare
R = Rank
R = Recommend

These verbs often tell you what the agent must do.

Example:

> Identify overdue tasks.

Problem:

> overdue tasks

> Detect suspicious transactions.

Problem:

> suspicious transactions

> Compare CVs with job requirements.

Problem:

> candidate suitability

So look for the **action verb** and what comes after it.

---

7. TASKS trick

Tasks are simply the steps the agent follows.

You can usually write:

Markdown

```
## Tasks
1. Read the supplied data.
2. Analyse the information.
3. Identify problems or risks.
4. Recommend appropriate actions.
5. Generate the required output.
```

Then modify it slightly for the scenario.

Example for students:

Markdown

```
## Tasks
1. Read student marks.
2. Check attendance.
3. Identify poor performance.
4. Identify students at risk.
5. Recommend academic support.
```

---

8. KNOWLEDGE BASE trick

Ask:

> **Which files contain information that helps the agent understand the situation?**

For your Project Tracking Agent, useful examples include:

```
Plain text
project_brief.md
weekly_update.md
risk_register.md
stakeholder_decisions.md
meeting_notes.md
```

The Custom GPT guide also recommends supporting knowledge such as past reports, project scope information and organizational rules. 1_Custom_GPT_Build_Guide (1)

Very important

Knowledge files ≠ input data automatically.

For example:

`tasks_template.csv`

contains structured task data.

But:

`risk_register.md`

contains supporting knowledge.

---

9. If they say “add 2–3 entries”

You could add files like:

Markdown

```
# Risk Register

## Risk 1
Task: T003
Risk: Software licence expired
Impact: High
Action: Renew the licence.

## Risk 2
Task: T004
Risk: Data migration may be delayed
Impact: High
Action: Complete T003 first.
```

Another:

Markdown

```
# Stakeholder Decisions

## Decision 1
Task: T001
Decision: Sponsor must approve the project scope.

## Decision 2
Task: T003
Decision: Finance must approve the licence payment.
```

Keep it **short, clear and relevant**.

---

10. Markdown `.md` you need to know

You do NOT need complicated Markdown.

Main heading:

Markdown

```
# Project Tracking Agent
```

Subheading:

Markdown

```
## Role
```

Bullet:

```
Markdown
- Risk 1
- Risk 2
```

Numbering:

```
Markdown
1. Read the data.
2. Analyse the data.
3. Generate the report.
```

Bold:

```
Markdown
**Important**
```

That's enough for many practical tasks.

---

11. Master `.md` template

Memorise this structure:

```
Markdown
# Agent Name

## Role
You are a [type] agent.

## Goal
Your goal is to [main goal].

## Inputs
Use:
- [data 1]
- [data 2]
```

```
## Tasks
1. Read the supplied data.
2. Analyse the information.
3. Identify problems.
4. Recommend actions.
5. Generate the output.
```

You change only the words in brackets.

---

12. GUARDRAIL trick

If you don't know what guardrail to write, these are safe:

> Do not invent missing information.

> Use only the supplied information.

> Protect private information.

> Clearly state when information is missing.

> Escalate uncertain or high-risk cases to a human.

Unit 4 specifically discusses relevance checks, safety checks, PII/privacy protection, moderation, tool safeguards and output validation. Unit 4 - A-Practical-Guide-to-B…

---

13. Human intervention

If asked:

> When should the agent transfer control to a human?

Easy answer:

> When the agent is uncertain, repeatedly fails, or when the action is high risk.

Unit 4 specifically recommends human intervention for failed attempts and sensitive/high-risk actions. Unit 4 - A-Practical-Guide-to-B…

---

14. “Specify the output you need”

This does NOT mean:

> I need a report.

Be specific.

Use:

> **WHO/WHAT + STATUS + REASON + ACTION**

For project tracking:

```
Output needed:

- Task ID
- Task owner
- Status
- Reason for status
- Risks/blockers
- Recommended action
```

For students:

```
- Student ID
- Performance
- Attendance
- Risk status
- Reason
- Recommended support
```

For recruitment:

```
- Candidate name
- Matching skills
- Missing requirements
- Suitability
- Reason
- Recommendation
```

If you panic, use:

> **ID/Name + Status + Reason + Recommended Action**

It works almost everywhere.

---

15. Report type trick

If they ask what kind of report should be generated:

> **TOPIC + REPORT**

Examples:

Scenario	Report
Project	Project status report
Student	Student performance report
Recruitment	Candidate screening report
Expenses	Expense compliance report
Inventory	Inventory status report
Risks	Risk assessment report
Sales	Sales performance report
Marketing	Campaign performance report
IT	Incident report
Vendors	Vendor performance report

---

16. Your TWO prompt templates

These are the two I want you to memorise.

Prompt 1 — Analyse

> Analyse the supplied **[DATA]** and identify **[PROBLEM]**. Explain the reason and provide **[ID/NAME,**

> **STATUS, REASON, RECOMMENDED ACTION]**. Do not invent missing information.

Prompt 2 — Report

> Generate a **[TOPIC] report** showing **[SUMMARY, MAIN FINDINGS, PROBLEMS/RISKS, RECOMMENDED ACTIONS]**. Use only the supplied files and recommend appropriate next actions.

You only change the brackets.

---

17. Example: Project scenario

Scenario:

> Identify tasks delaying the project.

Prompt 1:

> Analyse the supplied **project task register** and identify **overdue, blocked and critical tasks**. Explain the reason and provide the **task ID, owner, status, reason and recommended action**. Do not invent missing information.

Prompt 2:

> Generate a **project status report** showing the **overall status, overdue tasks, blockers, risks and recommended actions**. Use only the supplied files and recommend appropriate next actions.

---

18. Example: Student scenario

Prompt 1:

> Analyse the supplied **student marks and attendance records** and identify **students who may be academically at risk**. Explain the reason and provide the **student ID, status, reason and recommended support**. Do not invent missing information.

Prompt 2:

> Generate a **student performance report** showing **overall performance, attendance problems, students at risk and recommended support**. Use only the supplied files.

---

19. Example: Recruitment

Prompt 1:

> Analyse the supplied **CVs and job requirements** and identify **the most suitable candidates**. Explain the reason and provide the **candidate name, suitability, matching skills and recommendation**. Do not invent missing information.

Prompt 2:

> Generate a **candidate screening report** showing **matching skills, missing requirements, suitability and recommendations**.

---

20. What “Generate an output” probably means

In the practical, this could mean:

1. Run/open the agent.
2. Load/import information.
3. Add knowledge if required.
4. Give the agent a prompt or click **Generate Report**.
5. Let it produce the result.
6. Check that the result uses the supplied information.

Example generated output:

Plain text

```
Overall Status: RED

Critical Task:
T003 — Configure test environment

Reason:
The task is critical and blocked because the software licence expired.

Risk:
Data migration may be delayed.

Recommended Action:
Renew the software licence and complete T003 before continuing with T004.
```

The output should show that the agent **used the information you gave it**.

---

21. After adding knowledge entries

Suppose you add:

```
risk_register.md
stakeholder_decisions.md
```

Then ask something related to them:

> What are the main project risks and decisions required?

The answer should use those files.

That demonstrates that the knowledge base works.

---

22. Your Project Tracking Agent — how to run it

For the starter folder you were given:

```
py app.py
```

or:

```
python app.py
```

Then open:

```
http://127.0.0.1:8765
```

The README tells you to:

> Add demo tasks → Generate report.

The demo is expected to produce **Red** because a critical task is blocked and overdue.

---

23. Project Tracking Agent RAG rules

These are worth knowing because they are directly in your supplied starter.

RED

> Critical AND overdue
> OR
> Critical AND blocked

Memory:

> **RED = Critical problem**

AMBER

> overdue
> blocked
> or due within 7 days
> but not Red

Memory:

> **AMBER = Attention needed**

UNKNOWN

> latest information is older than 7 days and there is no Red/Amber problem.

Memory:

> **UNKNOWN = Old/stale information**

GREEN

> No configured risk/problem.

Memory:

> **GREEN = Good**

Overall status follows:

> **RED → AMBER → UNKNOWN → GREEN**

Meaning Red has the highest priority.

---

24. Understand the CSV

The starter uses:

```csv
id,task,owner,due,progress,critical,blocked,updated
```

Know these:

Column	Meaning
`id`	Task ID
`task`	Task name
`owner`	Responsible person
`due`	Deadline
`progress`	Completion %
`critical`	1 = critical
`blocked`	1 = blocked
`updated`	Last update

For `critical` and `blocked`:

```
1 = YES
0 = NO
```

---

25. Important starter-folder files

Know their purpose:

File	Purpose
`app.py`	Runs the agent/application

File	Purpose
`README.md`	Explains how to use it
`AGENT_INSTRUCTIONS.md`	Defines agent behaviour/rules
`tasks_template.csv`	Structured task data
`project_brief.md`	Project context
`weekly_update.md`	Progress updates
`knowledge_base/`	Supporting knowledge
`test_agent.py`	Tests the application

---

26. If asked “What is the role?”

For your starter:

> The role is to act as a **project progress and status reporting assistant**.

---

27. If asked “What is the goal?”

> The goal is to analyse project progress, identify risks/problems and generate a project status report.

---

28. If asked “What are the inputs?”

> Task register, project information, weekly updates and knowledge-base documents.

---

29. If asked “What is the output?”

> A structured project status report containing the overall status, task progress, blockers, risks, decisions required and proposed actions.

The supplied instructions explicitly require things such as executive summary, overall status, task progress, completed and overdue work, blockers, risks, decisions and proposed actions.

---

30. If asked “What is a tool?”

Simple answer:

> A tool allows the agent to retrieve information or perform an action.

Unit 4 separates tools into data/retrieval tools, action tools and orchestration tools.
Unit 4 - A-Practical-Guide-to-B…

Example:

> Searching the knowledge base is a data/retrieval tool.

---

31. Single agent vs multi-agent

If asked:

Single agent

> One agent handles the workflow using its instructions and tools.

Multi-agent

> Multiple specialised agents work together.

Use multiple agents when the workflow becomes too complex or tools/instructions become difficult for one agent to manage. Unit 4 - A-Practical-Guide-to-B…

Easy memory:

> **Simple = single**
> **Complex = multi**

---

32. Knowledge base vs tool vs instruction

Don't mix these up.

Knowledge

> Information the agent can use.

Example:

`risk_register.md`

Tool

> Something the agent uses to retrieve information or perform an action.

Example:

search knowledge base.

Instruction

> Rule telling the agent how to behave.

Example:

> Do not invent missing information.

---

33. Facts vs recommendations

A good agent should not mix them.

Example:

Fact:

> T003 is blocked.

Recommendation:

> Renew the software licence.

Don't write:

> Finance has approved the payment.

unless the files actually say that.

Your supplied Custom GPT instructions specifically say not to invent unsupported facts.

1_Custom_GPT_Build_Guide (1)

---

34. If information is missing

Do NOT invent.

Say:

> The information is not available in the supplied files.

Or:

> Clarification is required.

Or:

> Escalate to the responsible human.

---

35. If asked “Why is this an agent?”

Simple answer:

> It is an agent because it can use instructions and information to make decisions and complete a workflow toward a goal.

The guide describes agents as systems that can independently accomplish tasks and use tools while following guardrails. Unit 4 - A-Practical-Guide-to-B…

---

36. If asked “Why do we need clear instructions?”

Answer:

> Clear instructions tell the agent exactly what to do and reduce mistakes and confusion.

The Unit 4 guide explicitly says good instructions reduce ambiguity and improve decision-making.
Unit 4 - A-Practical-Guide-to-B…

---

37. If asked to improve a bad prompt

Bad:

> Check the project.

Better:

> Analyse the project task register and identify overdue, blocked and critical tasks. Provide the task ID, owner, status, reason and recommended action. Do not invent missing information.

Think:

> **Specific beats vague.**

---

38. Your emergency trick if you don't know the scenario

Write this:

Plain text

```
ROLE:
You are a [topic] agent.

GOAL:
Your goal is to analyse the supplied information and identify important problems.

DATA:
Use the supplied files and records.

TASK:
Analyse the information, identify problems and recommend actions.

GUARDRAILS:
Do not invent information.
Use only supplied data.
Escalate uncertain/high-risk cases.

OUTPUT:
Name/ID
Status
```

That can rescue you in many scenarios.

---

39. How I think the practical could be structured

My strongest prediction is something close to:

4.1 Open/test the agent and understand the files.

4.2 Identify appropriate knowledge files and add new entries.

4.3 Generate an output/report and possibly interpret it.

4.4 Specify the output you want and create two appropriate prompts.

The exact scenario may change, but the **thinking pattern stays the same**.

---

40. What to do immediately when you receive the practical

Use this exact order:

1. Read the **scenario first**.
2. Identify the **agent's role and goal**.
3. Look through the supplied folder.
4. Find the **README/instructions**.
5. Find the **data/input files**.
6. Find the **knowledge-base folder**.
7. Run/test the program if instructed.
8. Add only **relevant information**.
9. Generate an output.
10. Check whether the output makes sense.
11. For prompts, use your two memorised templates.
12. Never invent missing information.

Final things to memorise tonight

If you remember nothing else, remember these:

> **RGDTKGOS**
> Role → Goal → Data → Tasks → Knowledge → Guardrails → Output → Stop

> **IDAM-CRR**
> Identify → Detect → Analyse → Monitor → Compare → Rank → Recommend

> **Required fields:**
> **ID/Name → Status → Reason → Recommended Action**

> **Report:**
> **Topic + Report**

> **Guardrails:**
> **Don't invent → Use supplied data → Protect information → Escalate uncertainty**

And your two prompt formulas:

> **Analyse the supplied [DATA] and identify [PROBLEM]. Explain the reason and provide [ID/NAME, STATUS, REASON, RECOMMENDED ACTION]. Do not invent missing information.**

> **Generate a [TOPIC] report showing [SUMMARY, MAIN FINDINGS, PROBLEMS/RISKS, RECOMMENDED ACTIONS]. Use only the supplied files and recommend appropriate next actions.**

Those are the pieces I would prioritise for tomorrow's practical.